WARNING BUILD ERROR REPORT

.NET Error & Warning Fix Documentation TokensApp

Complete root-cause analysis, fixes, and step-by-step resolution for all issues encountered in the TokensApp project.

PROJECT	ISSUE 1	ISSUE 2
TokensApp	CS1024 Build Error	HTTPS Dev-Cert

TOTAL FIXED	DATE	STATUS
2 Issues	March 11, 2026	All Resolved

1 Project Overview

Property	Value
Project Name	TokensApp
Framework	.NET 8 / ASP.NET Core Web API
Language	C# 12
File Affected	`Controller/TokensController.cs`
Build Tool	MSBuild 17.8.49 (dotnet build)
Error Code	CS1024 — Preprocessor directive expected
Total Errors	8
Total Warnings	0

2 Build Errors Encountered

The following 8 compiler errors were produced when running `dotnet build`:

Error CS1024 — Line 1

```
TokensController.cs(1,1): error CS1024: Preprocessor directive expected
```

Offending line: ## --- Controllers\TokensController.cs ---

Error CS1024 — Line 16

```
TokensController.cs(16,1): error CS1024: Preprocessor directive expected
```

Offending line: ## --- API 1: GET All Vendor Categories ---

Error CS1024 — Line 29

```
TokensController.cs(29,1): error CS1024: Preprocessor directive expected
```

Offending line: ## --- API 2: GET Vendors (with optional category_id and city filter) ---

Error CS1024 — Line 56

```
TokensController.cs(56,1): error CS1024: Preprocessor directive expected
```

Offending line: ## --- API 3: GET Vendor Services ---

Error CS1024 — Line 77

```
TokensController.cs(77,1): error CS1024: Preprocessor directive expected
```

Offending line: ## --- API 4: POST Book Appointment (Create Token) ---

Error CS1024 — Line 113

```
TokensController.cs(113,1): error CS1024: Preprocessor directive expected
```

Offending line: ## --- API 5: GET Appointment by ID ---

Error CS1024 — Line 122

```
TokensController.cs(122,1): error CS1024: Preprocessor directive expected
```

Offending line: ## --- API 6: PATCH Cancel Appointment ---

Error CS1024 — Line 144

```
TokensController.cs(144,1): error CS1024: Preprocessor directive expected
```

Offending line: ## --- Request/Response DTOs ---

---

3 Root Cause Analysis

ROOT CAUSE

Markdown-formatted section headers (## --- Section Name ---) were accidentally embedded directly inside the C# source file TokensController.cs. These are not valid C# code and are not ignored by the compiler.

How It Happened

When the code was generated (by an AI assistant or copy-pasted from documentation), Markdown section headers were used as visual separators to label different API sections. These headers were intended as documentation but were inadvertently included directly in the `.cs` source file instead of being placed in a separate README or comment block.

Why the C# Compiler Fails

The C# compiler processes each line of a source file. When it encounters a line beginning with `#`, it expects a valid C# preprocessor directive (such as `#if`, `#region`, `#pragma`, etc.).

A Markdown heading like `## Some Title` starts with `#`, so the compiler interprets it as an attempted preprocessor directive. Since `##` is not recognized as any valid directive, it raises **CS1024: Preprocessor directive expected**.

TOKENSCONTROLLER.CS — BEFORE FIX (INVALID)

```
## --- Controllers\TokensController.cs ---          <-- INVALID
    using Microsoft.AspNetCore.Mvc;
    using Microsoft.EntityFrameworkCore;
    ... valid C# code ...
## --- API 1: GET All Vendor Categories ---          <-- INVALID
    [HttpGet("categories")]
    public async Task<IActionResult> GetCategories() { ... }
## --- API 2: GET Vendors ---                        <-- INVALID
    ... more valid C# code ...
```

Aspect	Details
Error Code	`CS1024`
Official Name	Preprocessor directive expected
Trigger	A line starts with # but is not a valid C# preprocessor directive

Valid C# directives	`#if`, `#else`, `#elif`, `#endif`, `#define`, `#undef`, `#warning`, `#error`, `#line`, `#region`, `#endregion`, `#pragma`, `#nullable`
Invalid usage found	Markdown heading `## --- ... ---` (not a C# directive)
Number of occurrences	8 lines in the file

4 Solution Applied

FIX APPLIED

All 8 Markdown header lines were removed from TokensController.cs. Section labels were converted to standard C# single-line comments (// --- ... ---), which are completely valid and serve the same visual purpose.

TOKENSCONTROLLER.CS — AFTER FIX (VALID)

```
// --- API 1: GET All Vendor Categories ----------------------
/// <summary>Get all active vendor categories</summary>
[HttpGet("categories")]
public async Task<IActionResult> GetCategories()
{
    var cats = await _db.VendorCategories
        .Where(c => c.IsActive) ...
    return Ok(cats);
}
// The // comment replaces ## header — valid C#, same readability
```

What Changed — Before vs After

Before (Invalid)	After (Valid)
`## --- Controllers\TokensController.cs ---`	*(removed — file-level annotation not needed in code)*
`## --- API 1: GET All Vendor Categories ---`	`// --- API 1: GET All Vendor Categories ---`
`## --- API 2: GET Vendors ... ---`	`// --- API 2: GET Vendors ... ---`

`## --- API 3: GET Vendor Services ---`	`// --- API 3: GET Vendor Services ---`
`## --- API 4: POST Book Appointment ---`	`// --- API 4: POST Book Appointment ---`
`## --- API 5: GET Appointment by ID ---`	`// --- API 5: GET Appointment by ID ---`
`## --- API 6: PATCH Cancel Appointment ---`	`// --- API 6: PATCH Cancel Appointment ---`
`## --- Request/Response DTOs ---`	`// --- Request/Response DTOs ---`

5 Step-by-Step Resolution Guide

1 Run dotnet build to capture errors

Open a terminal in the project directory and run `dotnet build`. Note all reported error lines — each CS1024 error includes the exact file name and line number.

2 Open the affected file in your editor

Open `Controller/TokensController.cs` (or whatever file MSBuild reports). Navigate to each reported line number.

3 Identify invalid lines

Look for lines starting with ## or any other Markdown syntax that was accidentally placed in the C# file. These lines are completely invalid in C# source code.

4 Remove or convert the invalid lines

Delete each Markdown header line, OR convert it to a C# comment by replacing ## with `//`. For example: `## --- API 1 ---` becomes `// --- API 1 ---`

5 Save the file

Save all changes to `TokensController.cs` in your editor.

6 Run dotnet build again to verify

Run `dotnet build` again. The expected output should be: **Build succeeded. 0 Error(s). 0 Warning(s).**

6 Verification — Build Output After Fix

✓

Build Succeeded — 0 Errors, 0 Warnings

After removing all 8 invalid Markdown lines, running `dotnet build` produced a clean successful build.

DOTNET BUILD — TERMINAL OUTPUT (AFTER FIX)

```
MSBuild version 17.8.49+7806cbf7b for .NET
  Determining projects to restore...
  All projects are up-to-date for restore.
  TokensApp -> ...\bin\Debug\net8.0\TokensApp.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:23.08
Exit code: 0
```

---

7 Prevention and Best Practices — CS1024

KEY RULE TO REMEMBER

In C# source files (.cs), NEVER use Markdown syntax. Only valid C# code, C# comments (// or /* */), and XML doc comments (/// summary) are permitted.

Scenario	Correct Approach
Add a visual section separator in C#	`// --- Section Name ---`
Document an API method	XML doc comments: `/// <summary>...</summary>`
Add project-level documentation	Create a `README.md` file at the project root
AI-generated code review	Always inspect AI-generated .cs files for accidental Markdown before committing

CI/CD prevention	Run `dotnet build` as the first step in your CI pipeline to catch these early

ISSUE 2 OF 2

HTTPS Developer Certificate Not Trusted Warning

Encountered when running `dotnet run`

8 Warning Overview

Property	Value
Warning Source	Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServer
Event ID	[8]
Command Triggered By	`dotnet run`
Severity	Warning (app still starts, but HTTPS is untrusted)
Affects	HTTPS endpoints (e.g., https://localhost:7178)
Fix Command	`dotnet dev-certs https --trust`
Status	Resolved — certificate trusted successfully

9 Warning Message

```
Warning — KestrelServer[8]
```

```
warn: Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServer[8] The ASP.NET Core developer certificate is not trusted. For information about trusting the ASP.NET Core developer certificate, see https://aka.ms/aspnet/https-trust-dev-cert.
```

App continues to start, but browsers and API clients will show SSL/security warnings on HTTPS requests.

---

10 Root Cause Analysis

ROOT CAUSE

When .NET creates a new Web API project, it generates a self-signed SSL certificate for local HTTPS development. Windows does not automatically trust self-signed certificates — it must be explicitly added to the system's Trusted Root Certificate Authorities store.

Why This Happens

When you run `dotnet new webapi` (or any HTTPS-enabled ASP.NET Core project for the first time on a machine), a local development certificate is created at:

CERTIFICATE LOCATION

```
Windows Certificate Store: CurrentUser\My (Personal)
Subject: CN=localhost
Issuer:  CN=localhost (self-signed)
Trust:   NOT in Trusted Root CAs  <-- cause of warning
```

Because the certificate is **self-signed** and not in the Trusted Root store, both Windows and browsers treat HTTPS connections to `localhost` as untrusted. Kestrel (the ASP.NET Core web server) detects this and emits the warning.

Aspect	Details
Certificate Type	Self-signed (not issued by a public CA)
Created By	.NET SDK during first project creation on a machine
Problem	Not in Windows Trusted Root Certificate Authorities store
Effect	Kestrel warns, browsers show "Connection not secure"

Does app crash?	No — it still starts and runs, warning only

11 Solution Applied

FIX APPLIED

Ran `dotnet dev-certs https --trust` which installed the self-signed localhost certificate into the Windows Trusted Root Certificate Authorities store. Output confirmed: "Successfully trusted the existing HTTPS certificate."

COMMAND & OUTPUT

```
> dotnet dev-certs https --trust

Trusting the HTTPS development certificate was requested.
A confirmation prompt will be displayed if the certificate
was not previously trusted. Click yes on the prompt to
trust the certificate.

Successfully trusted the existing HTTPS certificate.
Exit code: 0
```

Step-by-Step Resolution

1. **Open a terminal in your project directory**

   Open PowerShell or Command Prompt. You do NOT need to be in the project folder — this command is machine-wide.

2. **Run the trust command**

   `dotnet dev-certs https --trust` — This locates your local .NET developer SSL certificate and adds it to the Windows Trusted Root store.

3. **Accept the Windows security prompt**

   A Windows UAC / security dialog will appear asking "Do you want to install this certificate?". Click **Yes** to trust it.

4. **Verify — run dotnet run again**

Run `dotnet run` again. The Kestrel warning about the untrusted certificate will no longer appear. Your app starts cleanly on both http and https.

12 Verification — dotnet run After Fix

✓

Warning Resolved — App Starts Cleanly

After running `dotnet dev-certs https --trust`, no certificate warnings appear. App listens on both HTTP and HTTPS.

DOTNET RUN — TERMINAL OUTPUT (AFTER FIX)

```
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7178
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5241
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
// No Kestrel [8] warning -- certificate is now trusted
```

13 Prevention and Best Practices — HTTPS Certificate

IMPORTANT NOTE

The command `dotnet dev-certs https --trust` only needs to be run ONCE per machine. It is a one-time setup step for any new .NET development environment.

Scenario	Recommendation
New developer machine setup	Run `dotnet dev-certs https --trust` as part of your onboarding checklist

Certificate expired or corrupted	Run `dotnet dev-certs https --clean` then `dotnet dev-certs https --trust`
Check certificate status	Run `dotnet dev-certs https --check`
Multiple .NET apps on same machine	Trust only once — it applies to all ASP.NET Core apps on the machine
Production environments	Never use self-signed dev certs in production; use proper CA-issued certificates

TokensApp — Error & Warning Fix Documentation

Issue 1: CS1024 Build Error (8 Errors Fixed) | Issue 2: HTTPS Dev-Cert Warning (Resolved) | March 11, 2026 | Generated by Antigravity AI